Spark Structured Streaming + Kafka Workshop

Beginner In-Class Team Task - Stream Processing Course

Workshop Overview

Duration	1 hour 30 minutes
Team size	Less than or equal 5 students
Main idea	Kafka produces order events. Spark reads them as a streaming consumer and performs simple analysis.
Difficulty level	Beginner - first exposure to Spark
Submission	Code + short explanation + screenshot of output

Important: This is a guided workshop. Do not worry if you do not understand all Spark internals today. Focus on the pipeline: Producer -> Kafka topic -> Spark consumer -> live output.

Learning Objectives

- Understand the role of Kafka as a message broker in a streaming pipeline.
- Understand the role of Spark Structured Streaming as a consumer and processor.
- Read streaming data from a Kafka topic using Spark.
- Parse JSON order events into columns.
- Apply simple streaming transformations such as select, filter, and aggregation.

Scenario

You are working with a small e-commerce system. Each time a customer places an order, the application sends an event to Kafka. Your team will make Spark consume these events and analyze them in real time.

Mini Architecture

```
Python Kafka Producer  ->  Kafka Topic: orders  ->  Spark Structured Streaming Consumer  ->  Console Output
```

Suggested Time Plan

0-10 min	TA explains the pipeline and tools.
10-25 min	Run Kafka and create the orders topic.
25-45 min	Run the producer and send sample orders.
45-75 min	Run Spark consumer and complete Tasks 1-4.
75-90 min	Bonus task + team discussion + screenshots.

Part A - Preparation

Required Tools

- Docker Desktop or a running Kafka environment
- Python 3
- PySpark
- kafka-python package
- Java installed if Spark requires it on your machine

TA Note: If setup time is limited, prepare Kafka and Spark before class. Students can focus on the producer and Spark code instead of installation.

Option 1 - Simple Kafka using Docker Compose

Create a file named docker-compose.yml and use the following content:

```
services:
  zookeeper:
    image: confluentinc/cp-zookeeper:7.5.3
    container_name: zookeeper
    environment:
      ZOOKEEPER_CLIENT_PORT: 2181
      ZOOKEEPER_TICK_TIME: 2000
    ports:
      - "2181:2181"

  kafka:
    image: confluentinc/cp-kafka:7.5.3
    container_name: kafka
    depends_on:
      - zookeeper
    ports:
      - "9092:9092"
    environment:
      KAFKA_BROKER_ID: 1
      KAFKA_ZOOKEEPER_CONNECT: zookeeper:2181
      KAFKA_LISTENERS: PLAINTEXT://0.0.0.0:9092
      KAFKA_ADVERTISED_LISTENERS: PLAINTEXT://localhost:9092
      KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 1
```

Start Kafka:

```
docker compose up -d
```

Create the Kafka topic:

```
docker exec -it kafka kafka-topics --create --topic orders --bootstrap-server localhost:9092 --partitions 1 --replication-factor 1
```

Check that the topic exists:

```
docker exec -it kafka kafka-topics --list --bootstrap-server localhost:9092
```

Part B - Kafka Producer

Install the producer package if needed:

```
pip install kafka-python
```

Create a file named producer_orders.py:

```
from kafka import KafkaProducer
import json
import time

producer = KafkaProducer(
    bootstrap_servers="localhost:9092",
    value_serializer=lambda v: json.dumps(v).encode("utf-8")
)

orders = [
    {"customer": "Ali", "product": "Laptop", "price": 1200},
    {"customer": "Sara", "product": "Phone", "price": 800},
    {"customer": "Ali", "product": "Mouse", "price": 25},
    {"customer": "Mona", "product": "Laptop", "price": 1100},
    {"customer": "Sara", "product": "Keyboard", "price": 75},
    {"customer": "Omar", "product": "Phone", "price": 650}
]

for order in orders:
    producer.send("orders", value=order)
    print("Sent:", order)
    time.sleep(2)

producer.flush()
producer.close()
```

Run the producer:

```
python producer_orders.py
```

Part C - Spark Consumer

Create a file named spark_kafka_consumer.py:

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import col, from_json
from pyspark.sql.types import StructType, StructField, StringType, IntegerType

spark = SparkSession.builder \
    .appName("KafkaSparkOrdersWorkshop") \
    .getOrCreate()

spark.sparkContext.setLogLevel("WARN")

schema = StructType([
    StructField("customer", StringType(), True),
    StructField("product", StringType(), True),
    StructField("price", IntegerType(), True)
])

raw_df = spark.readStream \
```

```
    .format("kafka") \
    .option("kafka.bootstrap.servers", "localhost:9092") \
    .option("subscribe", "orders") \
    .option("startingOffsets", "latest") \
    .load()

json_df = raw_df.selectExpr("CAST(value AS STRING) as json_value")

orders_df = json_df.select(from_json(col("json_value"), schema).alias("data")) \
    .select("data.*")

# Task code will be written below this line

query = orders_df.writeStream \
    .outputMode("append") \
    .format("console") \
    .option("truncate", False) \
    .start()

query.awaitTermination()
```

Important: Run the Spark consumer first, then run the producer in another terminal. The Spark output should appear continuously in the consumer terminal.

Run Spark with Kafka Package

Use this command to run the Spark consumer:

```
spark-submit --packages org.apache.spark:spark-sql-kafka-0-10_2.12:3.5.1 spark_kafka_consumer.py
```

Version Note: The package version should match your Spark version. If your Spark version is different, ask your TA which version to use.

Part D - Student Tasks

Task 1 - Show Incoming Orders

Goal: Display all incoming orders from Kafka in the Spark console.

Expected columns: customer, product, price

Hint: Use orders_df directly in writeStream.

Task 2 - Select Product Names Only

Goal: Modify the Spark code to display only the product column.

```
products_df = orders_df.select("product")
```

Then use products_df in writeStream instead of orders_df.

Task 3 - Filter Expensive Orders

Goal: Display only orders where price is greater than 500.

```
expensive_df = orders_df.filter(col("price") > 500)
```

Question: Which orders appeared? Why did the Mouse and Keyboard orders disappear?

Task 4 - Add Discounted Price

Goal: Add a new column called discounted_price after applying a 10% discount.

```
discount_df = orders_df.withColumn("discounted_price", col("price") * 0.9)
```

Expected idea: Laptop with price 1200 becomes 1080.

Bonus Task - Count Orders per Customer

Goal: Count how many orders each customer placed.

```
count_df = orders_df.groupBy("customer").count()

query = count_df.writeStream \
    .outputMode("complete") \
    .format("console") \
    .option("truncate", False) \
    .start()

query.awaitTermination()
```

Why complete mode? For aggregation, Spark needs to print the updated full result table each time new data arrives.

Task Requirements:

- Final Spark consumer code
- Screenshot of producer sending events
- Screenshot of Spark console output
- Short explanation: What is Kafka doing? What is Spark doing?
- Team member names

Discussion Questions

1. Why do we need Kafka between the application and Spark?
2. What is the difference between batch data and streaming data?
3. What happens if the producer sends more orders while Spark is still running?
4. Which part of the pipeline is the producer? Which part is the consumer?

Common Problems and Fixes

No output in Spark	Start Spark consumer first, then run the producer again.
Kafka connection error	Check that Kafka container is running and port 9092 is available.
Package not found	Check that the Spark Kafka package version matches your Spark version.
Topic does not exist	Create the orders topic again using kafka-topics command.
Too many logs	Use spark.sparkContext.setLogLevel("WARN").

Final Reminder: The aim is not to memorize code. The aim is to understand the streaming flow: events are produced, Kafka stores them temporarily, and Spark consumes and processes them.